Wolfram Tools

Complete Comparison Guide & Mathematica Tutorial

For First-Time Users

Part 1: Which Tool Should I Use?

Part 2: Mathematica from Zero to Hero

PART 1

Wolfram Tool Comparison Charts

Quick Decision Flowchart

Use this simple flowchart to pick the right tool:

What do you need to do?

↓

Quick calculation or fact lookup? → **Wolfram Alpha (free)**

Step-by-step homework help? → **Wolfram Alpha Pro ($)**

Serious programming/research? → **Mathematica**

No software install, browser only? → **Wolfram Cloud (free tier)**

Share interactive content with students? → **Wolfram Cloud + Player**

Integrate with Python/other languages? → **Wolfram Engine (free for dev)**

Detailed Feature Comparison

Feature	Alpha	Alpha Pro	Cloud	Mathematica	Engine
Cost (All free 4 HCC)	Free	$60/yr	Free tier	License*	Free (dev)
Installation Required	—	—	—	✓	✓
Natural Language Input	✓	✓	✓	✓	—
Full Programming	—	—	✓	✓	✓
Step-by-Step Solutions	—	✓	—	—	—
Create Notebooks	—	—	✓	✓	—
Deploy Web Apps/APIs	—	—	✓	—	—
Call from Python/Java	—	—	—	✓	✓
Offline Use	—	—	—	✓	✓
Best For	Quick answers	Homework	Sharing	Research	Integration

** Mathematica: HCC has a site license*

Use Case Scenarios

Here's the right tool for specific tasks:

I want to...	Best Tool	Why
Calculate a quick integral	Wolfram Alpha	Free, instant, no setup
See step-by-step derivative solution	Alpha Pro	Shows learning steps
Look up a chemical compound	Wolfram Alpha	Built-in knowledge base
Solve a homework problem	Alpha Pro	Step-by-step explanations
Write a simulation program	Mathematica	Full programming environment
Create an interactive demo	Mathematica	Manipulate function
Share demo with students (no license)	Cloud + Player	Free viewing with Player
Try Wolfram Language without installing	Wolfram Cloud	Browser-based, free tier
Use Wolfram from Python	Wolfram Engine	Free, designed for integration
Collaborate on a notebook remotely	Wolfram Cloud	Built-in sharing
Build and deploy a web calculator	Wolfram Cloud	Deploy as API/form
Publish research with computations	Mathematica	Publication-quality output
Model physical/engineering systems	SystemModeler	Specialized for this

Computational Neuroscience: Which Tool?

For computational neuroscience specifically:

Task	Recommended	Alternative
Quick calculation (time constant, Nernst potential)	Wolfram Alpha	—
Solve Hodgkin-Huxley equations symbolically	Mathematica	Wolfram Cloud
Simulate single neuron models	Mathematica	Wolfram Cloud
Interactive parameter exploration (Manipulate)	Mathematica	Cloud (limited)
Large network simulations (1000+ neurons)	Mathematica + Compile	Consider NEST/Brian2
Signal processing (FFT, filtering)	Mathematica	Wolfram Cloud
Classroom demonstrations	Mathematica → Cloud	CDF + Player
Combine with Python (e.g., NumPy data)	Wolfram Engine	ExternalEvaluate

Recommendation for Students

Start with Wolfram Alpha for quick checks, then move to Wolfram Cloud (free) to learn the language. When ready for serious work, check if your school has Mathematica.

PART 2

Mathematica Tutorial for Complete Beginners

Lesson 1: Your First 10 Minutes

1.1 Opening Mathematica

When you open Mathematica, you'll see a blank notebook—think of it like a smart notepad that can do math.

1. Launch Mathematica (or go to wolframcloud.com and create a free account)
2. You'll see a blank white area—this is your notebook
3. Click anywhere and start typing
4. Press Shift+Enter to run what you typed

1.2 Your Very First Calculation

Type this and press Shift+Enter:

```
2 + 2
```

You should see:

```
4
```

Congratulations! You just used Mathematica. Now try these:

```
10 * 5       (* multiplication *)
100 / 4      (* division *)
2^10         (* exponentiation: 2 to the power of 10 *)
Sqrt[16]     (* square root *)
```

Important Syntax Rule

Functions use SQUARE BRACKETS [ ], not parentheses. Write Sqrt[16], not Sqrt(16). This is the #1 beginner mistake!

1.3 Understanding the Interface

Your notebook has two types of content:

- Input cells (In[1]:=) — What you type
- Output cells (Out[1]=) — What Mathematica returns

Each time you press Shift+Enter, the cell number increases. You can refer to previous outputs:

```
(* If your last output was 4 *)
% + 10       (* Returns 14 — % means "last output" *)
%% + 10      (* Returns previous-previous output + 10 *)
```

Lesson 2: The Three Golden Rules

Master these three rules and you'll avoid 90% of beginner errors:

Rule 1: Square Brackets for Functions

```
(* CORRECT *)
Sin[0.5]
Cos[Pi]
Log[100]
Exp[1]

(* WRONG - will cause errors *)
Sin(0.5)
Cos(Pi)
```

Parentheses ( ) are ONLY for grouping, like in math: (2 + 3) * 4

Rule 2: Curly Braces for Lists

```
(* Lists use curly braces *)
myList = {1, 2, 3, 4, 5}

(* A matrix is a list of lists *)
myMatrix = {{1, 2}, {3, 4}}

(* WRONG *)
myList = [1, 2, 3]    (* This won't work! *)
```

Rule 3: Capital Letters Matter

```
(* Built-in functions start with capitals *)
Sin[x]      (* Correct: the sine function *)
sin[x]      (* Wrong: Mathematica thinks this is YOUR function named "sin" *)

Pi          (* Correct: the constant π = 3.14159... *)
pi          (* Wrong: just a variable named "pi" with no value *)
```

> **Memory Trick**
> Wolfram Language = Square brackets for functions, Curly braces for lists, Capital letters for built-ins.

Lesson 3: Variables and Assignments

3.1 Storing Values

Use = to assign a value to a variable:

```
x = 5
y = 10
x + y       (* Returns 15 *)
```

Variable names can be anything (starting with a letter):

```
temperature = 98.6
patientAge = 42
myFavoriteNumber = 7
```

3.2 Clearing Variables

Variables persist until you clear them:

```
x = 5
x           (* Returns 5 *)

Clear[x]
x           (* Now returns just "x" — it's a symbol again *)
```

To clear everything and start fresh:

```
Clear["Global`*"]    (* Clears all your variables *)
```

3.3 Immediate (=) vs Delayed (:=) Assignment

This is important! There are two ways to assign:

```
(* Immediate assignment: = *)
a = RandomReal[]     (* a gets ONE random number, fixed forever *)
a                    (* Always the same number *)
a                    (* Still the same *)

(* Delayed assignment: := *)
b := RandomReal[]    (* b is RE-EVALUATED each time *)
b                    (* New random number *)
b                    (* Different random number *)
```

Use := when you want something recalculated each time (like functions).

Lesson 4: Creating Your Own Functions

4.1 Basic Function Definition

To create a function, use := and put an underscore after the parameter:

```
(* Define a function that squares its input *)
square[x_] := x^2

(* Use it *)
square[5]      (* Returns 25 *)
square[10]     (* Returns 100 *)
```

The underscore _ is crucial—it means "match any input":

```
f[x_] := x + 1     (* x_ matches anything *)
f[5]               (* Returns 6 *)
f[100]             (* Returns 101 *)
f[y]               (* Returns y + 1 *)
```

Common Mistake

Forgetting the underscore: f[x] := x^2 won't work as expected. Always write f[x_] := x^2

4.2 Functions with Multiple Inputs

```
(* Two inputs *)
add[a_, b_] := a + b
add[3, 7]          (* Returns 10 *)

(* Three inputs *)
weightedAverage[x_, y_, w_] := w*x + (1-w)*y
weightedAverage[100, 50, 0.3]   (* Returns 65 *)
```

4.3 A Practical Example: Neuron Activation

```
(* Sigmoid activation function *)
sigmoid[x_] := 1 / (1 + Exp[-x])

(* Test it *)
sigmoid[0]       (* Returns 0.5 — halfway point *)
sigmoid[10]      (* Returns ~1 — saturated *)
sigmoid[-10]     (* Returns ~0 — saturated other way *)

(* Plot it *)
Plot[sigmoid[x], {x, -10, 10}]
```

Lesson 5: Working with Lists

Lists are fundamental—neural data, time series, matrices—everything is lists.

5.1 Creating Lists

```
(* Manual list *)
data = {1, 4, 9, 16, 25}

(* Generate a sequence *)
Range[10]              (* {1, 2, 3, ..., 10} *)
Range[0, 1, 0.1]       (* {0, 0.1, 0.2, ..., 1.0} *)
Range[10, 1, -1]       (* {10, 9, 8, ..., 1} countdown *)

(* Generate using a formula *)
Table[n^2, {n, 1, 5}]  (* {1, 4, 9, 16, 25} *)
```

5.2 Accessing Elements

Use double square brackets [[ ]] to access elements:

```
data = {10, 20, 30, 40, 50}

data[[1]]       (* First element: 10 — indexing starts at 1! *)
data[[3]]       (* Third element: 30 *)
data[[-1]]      (* Last element: 50 *)
data[[-2]]      (* Second to last: 40 *)
data[[2;;4]]    (* Elements 2 through 4: {20, 30, 40} *)
```

5.3 Operations on Lists

```
data = {1, 2, 3, 4, 5}

(* Math operations work element-wise *)
data + 10        (* {11, 12, 13, 14, 15} *)
data * 2         (* {2, 4, 6, 8, 10} *)
data^2           (* {1, 4, 9, 16, 25} *)

(* Statistics *)
Total[data]              (* 15 — sum *)
Mean[data]               (* 3 — average *)
StandardDeviation[data]  (* ~1.58 *)
Max[data], Min[data]     (* 5, 1 *)
```

5.4 Applying Functions to Lists

Use /@ (shorthand for Map) to apply a function to every element:

```
data = {0, Pi/4, Pi/2, Pi}

Sin /@ data      (* Apply Sin to each: {0, 0.707..., 1, 0} *)

(* Equivalent longer form *)
Map[Sin, data]   (* Same result *)

(* With your own function *)
square[x_] := x^2
square /@ {1, 2, 3, 4}   (* {1, 4, 9, 16} *)
```

Lesson 6: Plotting and Visualization

Visualization is where Mathematica really shines.

6.1 Basic Plotting

```
(* Plot a function *)
Plot[Sin[x], {x, 0, 2*Pi}]

(* Plot multiple functions *)
Plot[{Sin[x], Cos[x]}, {x, 0, 2*Pi}]
```

6.2 Customizing Plots

```
Plot[Sin[x], {x, 0, 2*Pi},
  PlotStyle -> Red,                    (* Line color *)
  PlotLabel -> "Sine Wave",            (* Title *)
  AxesLabel -> {"Time (s)", "Amplitude"},  (* Axis labels *)
  GridLines -> Automatic,              (* Add grid *)
  PlotRange -> {-1.5, 1.5}             (* Y-axis range *)
]
```

6.3 Plotting Data Points

```
(* Your data *)
data = {1, 4, 2, 5, 3, 6, 4};

(* Just points *)
ListPlot[data]

(* Connected with lines *)
ListLinePlot[data]

(* With x-y pairs *)
xyData = {{0, 1}, {1, 4}, {2, 2}, {3, 5}, {4, 3}};
ListPlot[xyData, Joined -> True]
```

6.4 Combining Plots

```
(* Create two separate plots *)
p1 = Plot[Sin[x], {x, 0, 2*Pi}, PlotStyle -> Blue];
p2 = Plot[Cos[x], {x, 0, 2*Pi}, PlotStyle -> Red];

(* Combine them *)
Show[p1, p2]

(* Side by side *)
GraphicsRow[{p1, p2}]

(* Stacked vertically *)
GraphicsColumn[{p1, p2}]
```

Lesson 7: Solving Equations

7.1 Algebraic Equations (Solve)

```
(* Solve x^2 = 4 *)
Solve[x^2 == 4, x]       (* Note: == for equations, = for assignment *)
(* Returns: {{x -> -2}, {x -> 2}} *)

(* Quadratic equation *)
Solve[x^2 + 5*x + 6 == 0, x]
(* Returns: {{x -> -3}, {x -> -2}} *)

(* System of equations *)
Solve[{x + y == 10, x - y == 2}, {x, y}]
(* Returns: {{x -> 6, y -> 4}} *)
```

7.2 Numerical Solutions (NSolve, FindRoot)

```
(* When exact solution is hard *)
NSolve[x^5 - x - 1 == 0, x]   (* Numerical solutions *)

(* Find root near a guess *)
FindRoot[Cos[x] == x, {x, 0}]  (* Starts searching near x=0 *)
```

7.3 Differential Equations

This is where Mathematica excels for neuroscience!

```
(* Symbolic solution *)
DSolve[y'[x] == y[x], y[x], x]
(* Returns: y[x] -> C[1]*E^x *)

(* With initial condition *)
DSolve[{y'[x] == y[x], y[0] == 1}, y[x], x]
(* Returns: y[x] -> E^x *)

(* Numerical solution (for complex equations) *)
sol = NDSolve[{y'[x] == -y[x] + Sin[x], y[0] == 0}, y, {x, 0, 10}]
Plot[y[x] /. sol, {x, 0, 10}]
```

Lesson 8: Interactive Demonstrations

Manipulate lets you create interactive explorations with sliders.

8.1 Basic Manipulate

```
(* Interactive sine wave *)
Manipulate[
  Plot[A * Sin[f * x], {x, 0, 2*Pi}, PlotRange -> {-3, 3}],
  {A, 0.1, 3},    (* A slider from 0.1 to 3 *)
  {f, 1, 5}       (* f slider from 1 to 5 *)
]
```

When you run this, sliders appear! Move them to change A and f in real time.

8.2 Slider Options

```
Manipulate[
  Plot[Sin[f*x], {x, 0, 2*Pi}],

  (* Different slider styles *)
  {f, 1, 10, 1},           (* Step by 1: 1, 2, 3, ... *)
  {f, 1, 10, 0.5},         (* Step by 0.5 *)
  {{f, 5, "Frequency"}, 1, 10},  (* Default value 5, labeled *)
  {color, {Red, Blue, Green}}    (* Dropdown menu *)
]
```

8.3 Neuroscience Example: Explore Firing Rate

```
(* Interactive F-I curve explorer *)
Manipulate[
  Module[{rate},
    (* Simple firing rate model *)
    rate[I_] := If[I > threshold, (I - threshold)/tau, 0];

    Plot[rate[I], {I, 0, 10},
      PlotRange -> {{0, 10}, {0, 0.5}},
      AxesLabel -> {"Input Current", "Firing Rate"},
      PlotLabel -> StringForm["Threshold=``, τ=``", threshold, tau]
    ]
  ],
  {{threshold, 2, "Threshold"}, 0, 5},
  {{tau, 20, "Time Constant"}, 5, 50}
]
```

Lesson 9: Getting Help

9.1 Built-in Documentation

```
(* Get help on any function *)
?Sin              (* Quick summary *)
??Sin             (* Detailed information *)

(* Search for functions *)
?*Plot*           (* All functions containing "Plot" *)
```

9.2 The Documentation Center

Press F1 or go to Help → Documentation Center. This is your best friend—it has:

- Detailed explanations of every function
- Examples you can copy and modify
- Tutorials on specific topics
- "See Also" links to related functions

9.3 Wolfram Alpha Integration

Type = at the start of a cell to use natural language:

```
= integrate sin(x)^2 from 0 to pi

= solve x^2 + 3x - 4 = 0

= plot sin(x) and cos(x)
```

Mathematica converts your English to Wolfram Language code!

Lesson 10: Common Tasks Quick Reference

Task	Code
Square root	`Sqrt[x]`
Absolute value	`Abs[x]`
e^x	`Exp[x]`
Natural log	`Log[x]`
Log base 10	`Log10[x]`
Random number (0-1)	`RandomReal[]`
Random integer (1-10)	`RandomInteger[{1, 10}]`
List of 100 random numbers	`RandomReal[1, 100]`
Derivative	`D[f[x], x]`
Integral	`Integrate[f[x], x]`
Definite integral	`Integrate[f[x], {x, a, b}]`
Numerical value	`N[Pi, 20]  (* 20 digits of Pi *)`
Simplify expression	`Simplify[expr]`
Expand expression	`Expand[(x+1)^3]`
Factor polynomial	`Factor[x^2 - 1]`
Matrix multiply	`A . B`
Transpose	`Transpose[matrix]`
Inverse matrix	`Inverse[matrix]`
Eigenvalues	`Eigenvalues[matrix]`

Practice Exercises for Beginners

Try these to cement your understanding:

Exercise 1: Basic Calculations

Calculate: (a) 2^20, (b) √144, (c) sin(π/6), (d) e^2

Exercise 2: Create a Function

Write a function celsius[f] that converts Fahrenheit to Celsius. Test it with 32°F and 212°F.

Exercise 3: Work with Lists

Create a list of the first 10 perfect squares. Find their sum and mean.

Exercise 4: Plotting

Plot sin(x) and sin(2x) on the same graph from 0 to 2π, with different colors and a legend.

Exercise 5: Interactive Demo

Create a Manipulate that shows how changing amplitude and frequency affects a sine wave.

Solutions:

```
(* Exercise 1 *)
{2^20, Sqrt[144], Sin[Pi/6], Exp[2]}

(* Exercise 2 *)
celsius[f_] := (f - 32) * 5/9
{celsius[32], celsius[212]}  (* {0, 100} *)

(* Exercise 3 *)
squares = Table[n^2, {n, 1, 10}]
{Total[squares], Mean[squares]}

(* Exercise 4 *)
Plot[{Sin[x], Sin[2*x]}, {x, 0, 2*Pi},
  PlotStyle -> {Blue, Red},
  PlotLegends -> {"Sin[x]", "Sin[2x]"}]

(* Exercise 5 *)
Manipulate[
  Plot[A*Sin[f*x], {x, 0, 2*Pi}, PlotRange -> {-3, 3}],
  {{A, 1, "Amplitude"}, 0.1, 3},
  {{f, 1, "Frequency"}, 0.5, 5}
]
```

End of Guide

For more learning: Wolfram U (wolfram.com/wolfram-u) offers free courses

Wolfram Documentation Center (reference.wolfram.com) has comprehensive guides